

GitHub Repository Mining Project

Data Mining • PageRank • Apriori / FP-Growth • BERT

M1 Data Collection

M2 Data Engineering

M3 Pattern Mining

M4 Link Analysis

M5 BERT & Report



1. Project Overview

GitHub Repository Mining

This project analyzes software development trends by collecting repository data from GitHub. Students extract information such as repository names, programming languages, stars, forks, and topics, then build relationships between repositories based on shared technologies. The project integrates three main areas: **Web Mining**, **Association Rule Mining**, and **Link Analysis**, enhanced by **BERT-based deep learning** for text classification.

Component	Technique	Goal
Data collection	GitHub REST API	Repos: names, langs, stars, forks, topics
Pattern mining	Apriori / FP-Growth	Discover common tech-stack combinations
Link analysis	PageRank	Rank influential repositories
Visualization	NetworkX, Plotly	Charts, network graphs, dashboards
Text classification	BERT (fine-tuned)	Categorize repos by description & README

2. Grading Breakdown

Criterion	Owner	Marks
Data collection & understanding	Member 1	2
Data preprocessing	Member 2	2
Association rule mining	Member 3	2
Link analysis (PageRank)	Member 4	2
Visualization	Members 2 & 4	1
Report & presentation	Member 5	1
Task 1 subtotal		10
BERT model (Task 2)	Member 5	5
Total		15

3. Project Pipeline



4. Team Member Responsibilities

M1 Member 1 — Data Collection Lead

GitHub API integration, dataset design

1.1 GitHub API setup & authentication

Register a GitHub personal access token. Configure the PyGitHub or requests library and implement rate-limit handling (60 req/h unauthenticated → 5,000/h authenticated). Verify data quality before collection begins.

[\[PyGitHub\]](#) [\[requests\]](#) [\[GitHub REST API\]](#)

1.2 Repository data collection

Collect at least 1,000 repositories covering: full name, description, primary language, star count, fork count, topic tags, and README content. Target a diverse range of popular and niche repos.

[\[GitHub Search API\]](#) [\[pandas\]](#) [\[json\]](#)

1.3 Transaction dataset design

Transform raw repo data into transaction records where each repo is one transaction and its topics + language are items. This dataset feeds directly into Member 3's association rule mining.

[\[pandas\]](#) [\[mlxtend\]](#) [\[CSV\]](#)

1.4 Dataset documentation

Write a data dictionary, describe the collection methodology, note any gaps or biases, and hand off clean files to Member 2 for preprocessing.

[\[Markdown\]](#) [\[Jupyter Notebook\]](#)

Code snippet:

```
import requests, pandas as pd, time
TOKEN = "ghp_your_token"
headers = {"Authorization": f"token {TOKEN}"}

def fetch_repos(query="stars:>500", per_page=100, pages=10):
    repos = []
    for page in range(1, pages + 1):
        url = (f"https://api.github.com/search/repositories"
              f"?q={query}&per_page={per_page}&page={page}&sort=stars")
        r = requests.get(url, headers=headers).json()
        repos.extend(r.get("items", []))
    time.sleep(1) # respect rate limits
```

```

return repos

raw = fetch_repos()
df = pd.DataFrame([
    {"name": r["full_name"], "description": r["description"],
    "language": r["language"], "stars": r["stargazers_count"],
    "forks": r["forks_count"], "topics": r["topics"]}
    for r in raw])
df.to_csv("repos.csv", index=False)

```

M2

Member 2 — Data Engineering & Visualization

Preprocessing, graph construction, EDA charts

2.1 Data cleaning & preprocessing

Handle null values, normalize language names, deduplicate repositories, filter bot/fork accounts, and encode categorical features for downstream algorithms.

[\[pandas\]](#) [\[numpy\]](#) [\[scikit-learn\]](#)

2.2 Graph construction

Build a weighted undirected graph where nodes are repositories and edges represent shared topics or language overlap. Higher edge weight means more shared attributes. This graph is used by Member 4's PageRank.

[\[NetworkX\]](#) [\[scipy\]](#)

2.3 Exploratory data analysis charts

Language distribution bar chart, star/fork histograms, topic frequency word cloud, and correlation matrix between numeric features.

[\[matplotlib\]](#) [\[seaborn\]](#) [\[wordcloud\]](#)

2.4 Network visualization

Render the repo graph using a force-directed layout. Color nodes by primary language, size by star count. Export an interactive HTML version for the report.

[\[pyvis\]](#) [\[gephi\]](#) [\[plotly\]](#)

Code snippet:

```

import networkx as nx
from itertools import combinations

G = nx.Graph()
for _, row in df.iterrows():
    G.add_node(row["name"], stars=row["stars"],
    language=row["language"])

# Edges for repos sharing a topic
topic_to_repos = {}
for _, row in df.iterrows():
    for t in row["topics"]:

```

```
topic_to_repos.setdefault(t, []).append(row["name"])

for topic, repos in topic_to_repos.items():
    for a, b in combinations(repos, 2):
        if G.has_edge(a, b):
            G[a][b]["weight"] += 1
        else:
            G.add_edge(a, b, weight=1)
```

M3

Member 3 — Association Rule Mining

Apriori, FP-Growth, tech-stack discovery

3.1 Apriori algorithm

Apply Apriori on the transaction dataset. Tune min_support and min_confidence. Extract frequent itemsets representing common technology combinations (e.g. Python + ML + NLP).

[\[mlxtend\]](#) [\[apriori\]](#) [\[pandas\]](#)

3.2 FP-Growth implementation

Apply FP-Growth as a scalable alternative to Apriori. Compare runtimes and rule quality between both algorithms to select the best approach.

[\[mlxtend\]](#) [\[fp-growth\]](#) [\[time\]](#)

3.3 Rule evaluation & interpretation

Rank rules by lift, confidence, and support. Identify top tech stacks. Prune redundant and trivial rules. Highlight actionable insights for the report.

[\[pandas\]](#) [\[matplotlib\]](#) [\[mlxtend\]](#)

3.4 Association graph visualization

Build a bipartite or technology network graph where strong rules are edges between technologies. Node size encodes frequency; edge thickness encodes lift.

[\[NetworkX\]](#) [\[pyvis\]](#) [\[matplotlib\]](#)

Code snippet:

```
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules
from mlxtend.preprocessing import TransactionEncoder

te = TransactionEncoder()
te_arr = te.fit(transactions).transform(transactions)
df_enc = pd.DataFrame(te_arr, columns=te.columns_)

# Apriori
freq_ap = apriori(df_enc, min_support=0.05, use_colnames=True)
rules_ap = association_rules(freq_ap, metric="lift", min_threshold=1.2)

# FP-Growth (faster)
freq_fp = fpgrowth(df_enc, min_support=0.05, use_colnames=True)
```

```
rules_fp = association_rules(freq_fp, metric="lift", min_threshold=1.2)

top = rules_ap.sort_values("lift", ascending=False).head(10)
print(top[["antecedents", "consequents", "support", "confidence", "lift"]])
```

M4

Member 4 — Link Analysis & Ranking

PageRank, centrality analysis, ranking charts

4.1 PageRank implementation

Run PageRank on the repo graph built by Member 2. Experiment with damping factors (default $d = 0.85$). Rank all repositories by influence score.

[\[NetworkX\]](#) [\[scipy\]](#) [\[numpy\]](#)

4.2 Influential repo identification

Extract the top-30 most influential repositories. Analyze whether high PageRank correlates with star count, fork count, or topic diversity.

[\[pandas\]](#) [\[matplotlib\]](#) [\[seaborn\]](#)

4.3 Ranking charts & insights

Horizontal bar chart of top repos by PageRank score. Scatter plot of PageRank vs. stars, colored by programming language. Export as figures for the report.

[\[matplotlib\]](#) [\[plotly\]](#) [\[seaborn\]](#)

4.4 Centrality comparison

Compare PageRank with degree centrality and betweenness centrality to validate results and identify bridging repositories that connect different tech communities.

[\[NetworkX\]](#) [\[pandas\]](#)

Code snippet:

```
import networkx as nx
import pandas as pd

# Run PageRank on the graph G from Member 2
pr = nx.pagerank(G, alpha=0.85, weight="weight")

pr_df = pd.DataFrame.from_dict(pr, orient="index",
                                columns=["pagerank"])
pr_df = pr_df.merge(
    df[["name", "stars", "language"]],
    left_index=True, right_on="name"
).sort_values("pagerank", ascending=False)

# Centrality comparison
deg = nx.degree_centrality(G)
btw = nx.betweenness_centrality(G, weight="weight")
print(pr_df.head(20))
```

5.1 BERT fine-tuning for classification

Fine-tune bert-base-uncased on repository descriptions and README files. Labels: AI/ML, Web Development, Mobile, DevOps, Data Engineering, Other. Handle class imbalance with weighted loss.

[\[transformers\]](#) [\[PyTorch\]](#) [\[HuggingFace\]](#)

5.2 Model training & evaluation

Train/validation/test split (70/15/15). Track accuracy and F1-score per class. Use early stopping. Save the best checkpoint. Report confusion matrix.

[\[scikit-learn\]](#) [\[wandb\]](#) [\[CUDA / Google Colab\]](#)

5.3 Classification results integration

Tag each repository with its predicted category. Show category distribution. Compare BERT predictions with human-assigned GitHub topic labels.

[\[pandas\]](#) [\[matplotlib\]](#) [\[seaborn\]](#)

5.4 Final report & presentation

Compile all findings into a structured report covering methodology, results, visualizations, and conclusions. Prepare slides representing all team members' work.

[\[LaTeX / Word\]](#) [\[PowerPoint\]](#) [\[Canva\]](#)

Code snippet:

```
from transformers import (BertTokenizer,
BertForSequenceClassification, Trainer, TrainingArguments)
from datasets import Dataset

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained(
"bert-base-uncased", num_labels=6)

label_map = {"AI/ML":0, "Web Dev":1, "DevOps":2,
"Mobile":3, "Data Eng":4, "Other":5}

def tokenize(batch):
return tokenizer(batch["text"], padding="max_length",
truncation=True, max_length=256)

args = TrainingArguments(
output_dir="./bert-github", num_train_epochs=3,
per_device_train_batch_size=16, evaluation_strategy="epoch",
load_best_model_at_end=True)

trainer = Trainer(model=model, args=args,
```

```
train_dataset=train_ds, eval_dataset=val_ds)  
trainer.train()
```


5. Recommended Tools & Libraries

Layer	Tools	Purpose
Language	Python 3.10+	Primary implementation language
Data handling	pandas, numpy	Cleaning, manipulation, analysis
API access	PyGitHub, requests	GitHub data collection
Pattern mining	mlxtend (Apriori, FP-Growth)	Association rule discovery
Graph analysis	NetworkX, scipy	Graph construction & PageRank
Visualization	matplotlib, seaborn, plotly, pyvis	Charts & network diagrams
Deep learning	HuggingFace Transformers, PyTorch	BERT fine-tuning
Experiment track	Weights & Biases (wandb)	Model training monitoring
Notebooks	Jupyter Lab	Collaborative development
Version control	Git + GitHub	Code & data versioning

6. Suggested Timeline

Week	Phase	Owner(s)	Deliverable
1	Data collection & API setup	M1	repos.csv, transactions.csv
2	Preprocessing & graph construction	M2	clean_repos.csv, graph.gml
3	EDA charts & association rule mining	M2, M3	EDA plots, Apriori/FP rules
4	PageRank analysis & ranking charts	M4	pagerank_scores.csv, charts
5	BERT fine-tuning & evaluation	M5	bert-github/ checkpoint
6	Integration, report & presentation	All / M5 lead	Final PDF report + slides